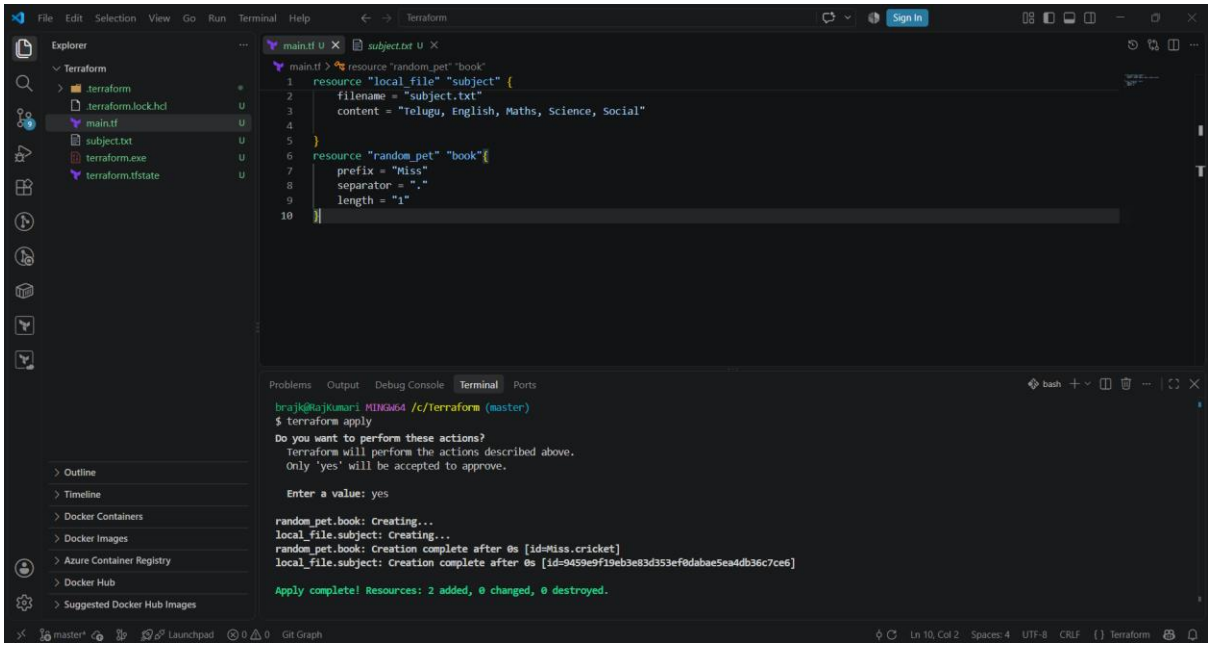
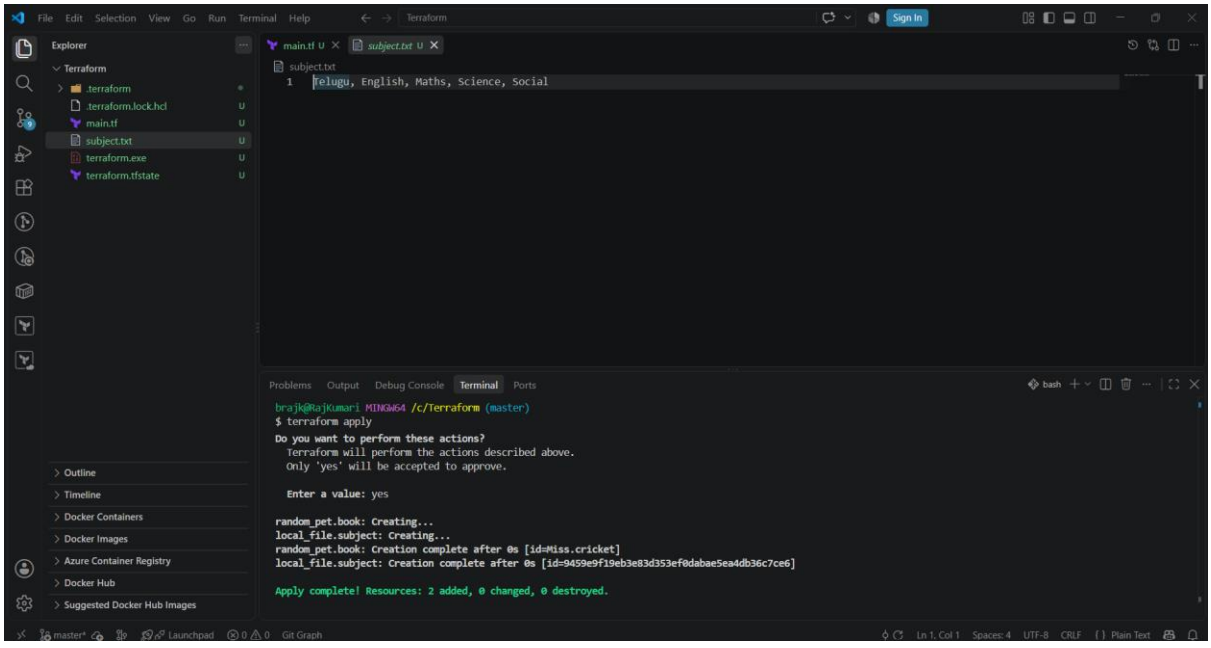
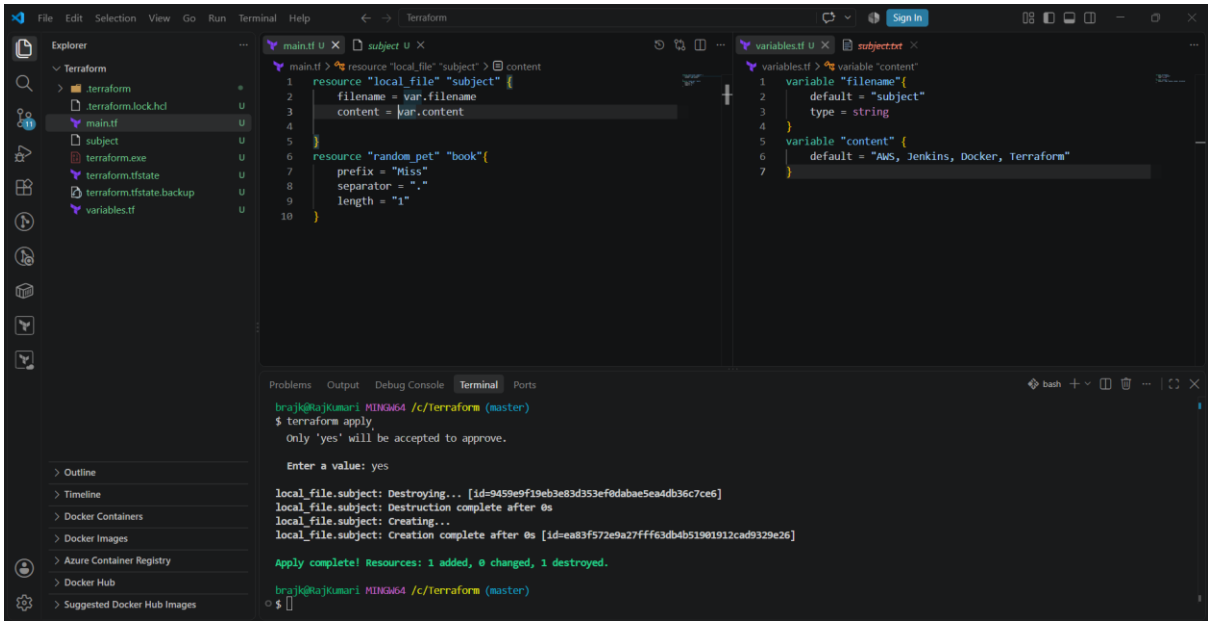
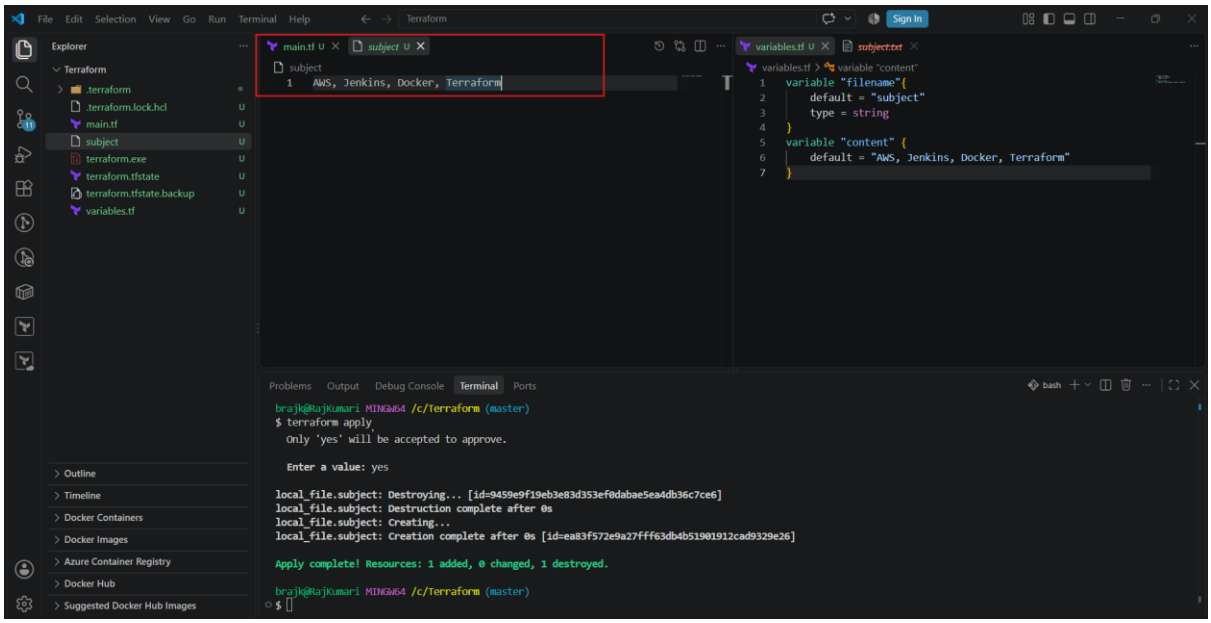
TERRAFORM TASK - 03

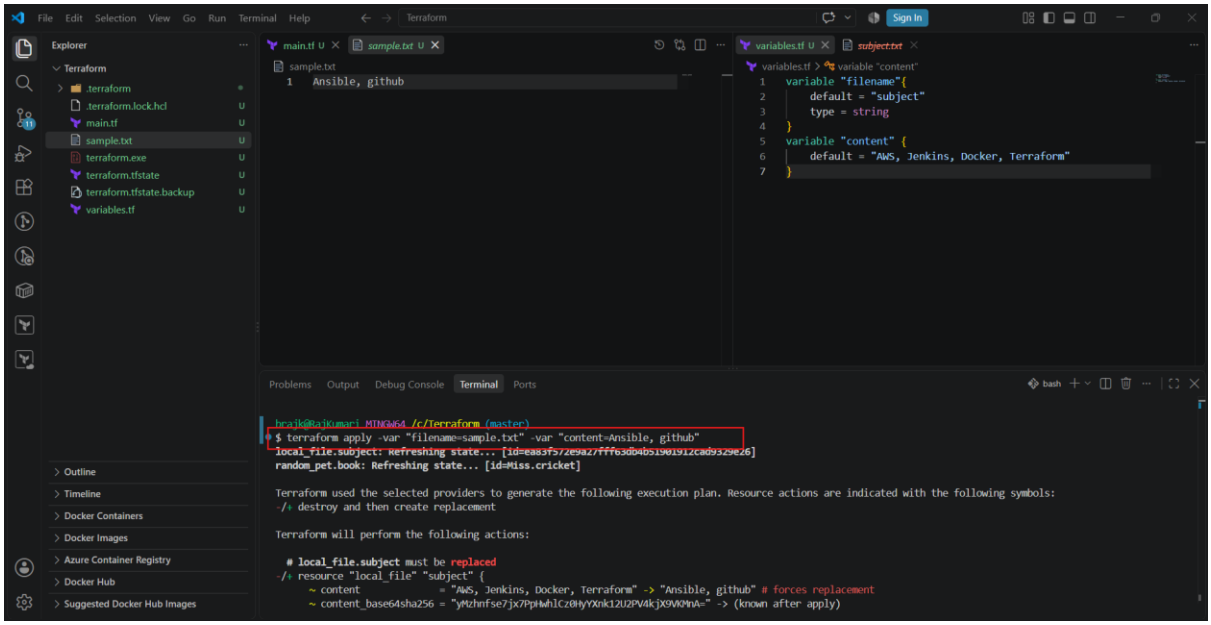
1.Execute the script shown in the video.






File Edit Selection View Go Run Terminal Help
Terraform
Sign In
Explorer
Terraform
.terraform
.terraform.lock.hcl
main.tf
subject
terraform.exe
terraform.tfstate
terraform.tfstate.backup
variables.tf
main.tf U
subject U
main.tf > resource "local_file" "subject" > content
resource "local_file" "subject" {
filename = var.filename
content = var.content
}
resource "random_pet" "book"{
prefix = "Miss"
separator = "."
length = "1"
}
variables.tf U
subject.txt
variables.tf > variable "content"
variable "filename"{
default = "subject"
type = string
}
variable "content" {
default = "AWS, Jenkins, Docker, Terraform"
}
Problems Output Debug Console Terminal Ports
bash
brajk@RajKumari MINGW64 /c/Terraform (master)
$ terraform apply
Only 'yes' will be accepted to approve.
Enter a value: yes
local_file.subject: Destroying... [id=9459e9f19eb3e83d353ef0dabae5ea4db36c7ce6]
local_file.subject: Destruction complete after 0s
local_file.subject: Creating...
local_file.subject: Creation complete after 0s [id=ea83f572e9a27fff63db4b51901912cad9329e26]
Apply complete! Resources: 1 added, 0 changed, 1 destroyed.
brajk@RajKumari MINGW64 /c/Terraform (master)
$
Outline
Timeline
Docker Containers
Docker Images
Azure Container Registry
Docker Hub
Suggested Docker Hub Images


File Edit Selection View Go Run Terminal Help
Terraform
Sign In
Explorer
Terraform
.terraform
.terraform.lock.hcl
main.tf
subject
terraform.exe
terraform.tfstate
terraform.tfstate.backup
variables.tf
main.tf U
subject U
subject
1 AWS, Jenkins, Docker, Terraform
variables.tf U
subject.txt
variables.tf > variable "content"
variable "filename"{
default = "subject"
type = string
}
variable "content" {
default = "AWS, Jenkins, Docker, Terraform"
}
Problems Output Debug Console Terminal Ports
bash
brajk@RajKumari MINGW64 /c/Terraform (master)
$ terraform apply
Only 'yes' will be accepted to approve.
Enter a value: yes
local_file.subject: Destroying... [id=9459e9f19eb3e83d353ef0dabae5ea4db36c7ce6]
local_file.subject: Destruction complete after 0s
local_file.subject: Creating...
local_file.subject: Creation complete after 0s [id=ea83f572e9a27fff63db4b51901912cad9329e26]
Apply complete! Resources: 1 added, 0 changed, 1 destroyed.
brajk@RajKumari MINGW64 /c/Terraform (master)
$
Outline
Timeline
Docker Containers
Docker Images
Azure Container Registry
Docker Hub
Suggested Docker Hub Images

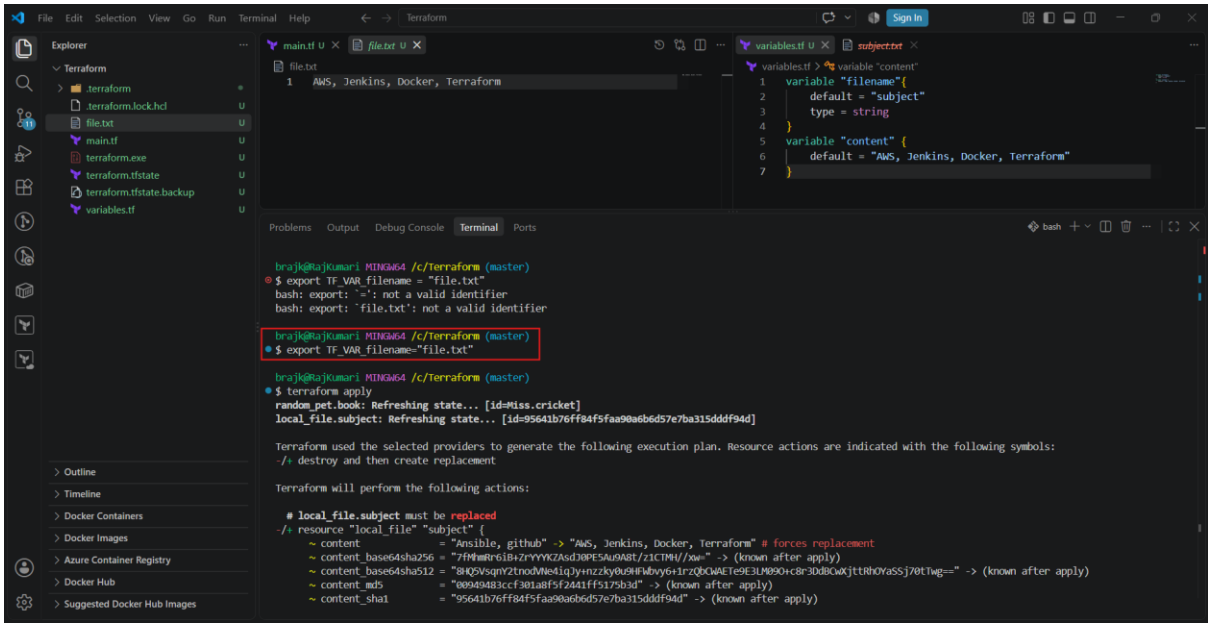


Environmental Variables in Terraform: Terraform environment variables are variables defined at the operating-system level and automatically consumed by Terraform using **the TF_VAR_ prefix to provide values** for Terraform input variables without hard-coding them in **.tf files.**

Why use them?

- Avoid hard-coding configuration values
- Different values can be used for different environments
- Useful in CI/CD pipelines such as Jenkins
- Can help keep sensitive values out of .tf files (though secrets should still be handled carefully)

Example: **export TF_VAR_filename="file.txt"**



Terraform Output Variable: A **Terraform output variable** is used to **display or expose information about resources created by Terraform** after terraform apply.

In simple words: **Output variables allow Terraform to show important values from your infrastructure, such as an EC2 instance IP address, DNS name, VPC ID, or S3 bucket name.**

Example

Suppose Terraform creates an EC2 instance:

```
resource "aws_instance" "server" {
  ami           = "ami-xxxxxxxx"
  instance_type = "t2.micro"
}
```

You can create an output variable:

```
output "instance_public_ip" {
  description = "Public IP address of the EC2 instance"
  value       = aws_instance.server.public_ip
}
```

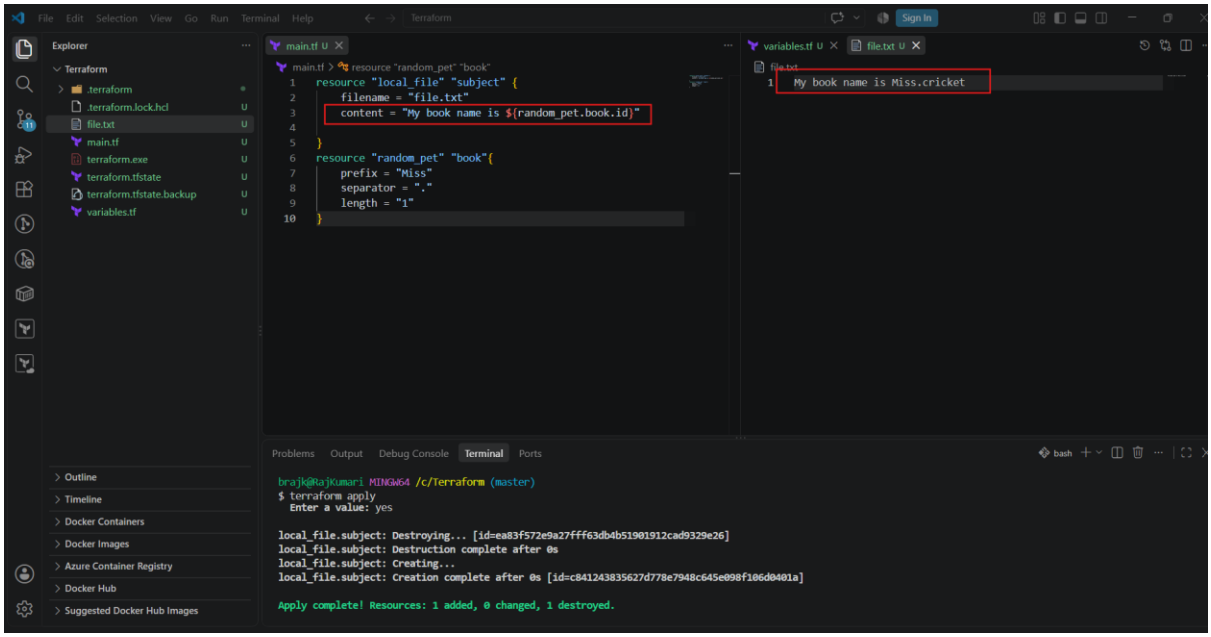
After running:

terraform apply

Terraform displays something like:

Apply complete! Resources: 1 added, 0 changed, 0 destroyed.

Outputs: instance_public_ip = "54.123.45.67"



2.Integrate Terraform in Jenkins using the Terraform plugin.

GitHub → Jenkins → Terraform Plugin → Terraform → AWS

One important point first: the Jenkins **Terraform plugin** is an older plugin (current listed version 1.0.10), and its documented configuration is primarily a **build wrapper**. For a modern Jenkins pipeline, you can still configure Terraform through **Manage Jenkins → Tools** and invoke Terraform from a Jenkinsfile. This is generally easier to maintain.

1. Overall architecture

Developer

|

| git push

v

GitHub Repository

|

v

Jenkins

|

| Terraform Plugin / Terraform installation

v

Terraform

|

| AWS credentials

v

AWS

|

+---- EC2

+---- VPC

+---- Security Group

+---- S3

+---- etc.

The important distinction is:

- **Jenkins Terraform plugin** → helps Jenkins integrate with Terraform.
- **Terraform AWS provider** → allows Terraform to create/manage AWS resources.

Terraform itself uses provider plugins to communicate with services such as AWS.

Step 2: Install Terraform on the Jenkins server

SSH into your Jenkins EC2 instance.

Check whether Terraform is already installed:

terraform version

If you get something like:

Terraform v1.x.x

you can proceed.

If not, install Terraform.

For example, on Amazon Linux:

sudo dnf update -y

Then install Terraform using HashiCorp's repository/package instructions appropriate for your Amazon Linux version.

After installation:

terraform version

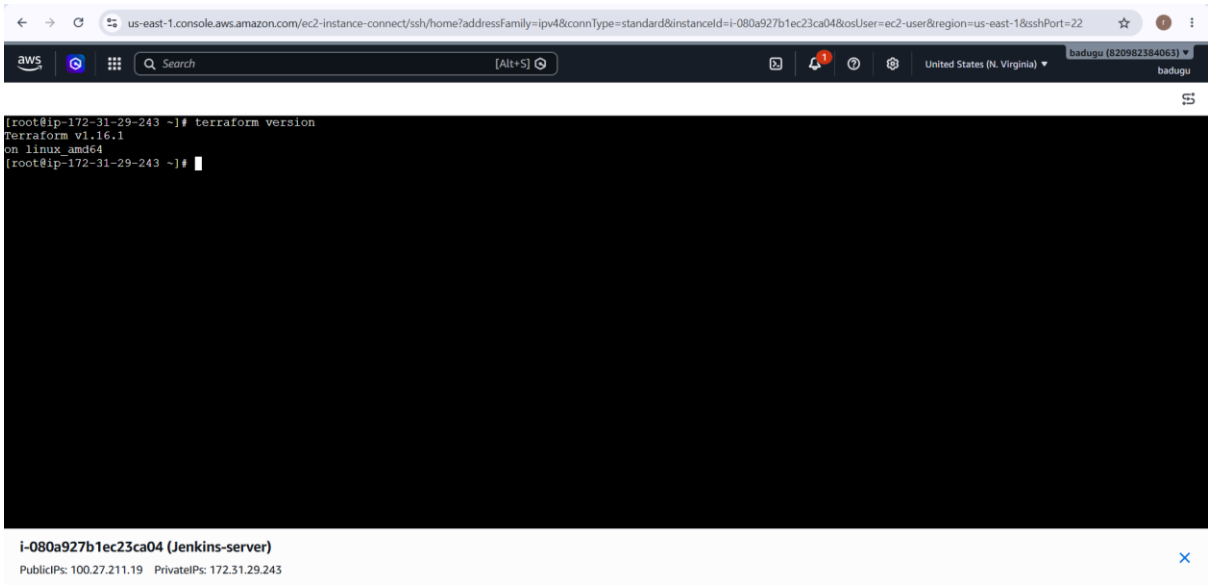
You should see:

Terraform v1.x.x

on linux_amd64

Why this is important

Jenkins ultimately needs access to the terraform executable on the machine/agent where the build runs.



Step 3: Install the Jenkins Terraform plugin

Go to:

Jenkins Dashboard

↓

Manage Jenkins

↓

Plugins

↓

Available plugins

Search:

Terraform

Install:

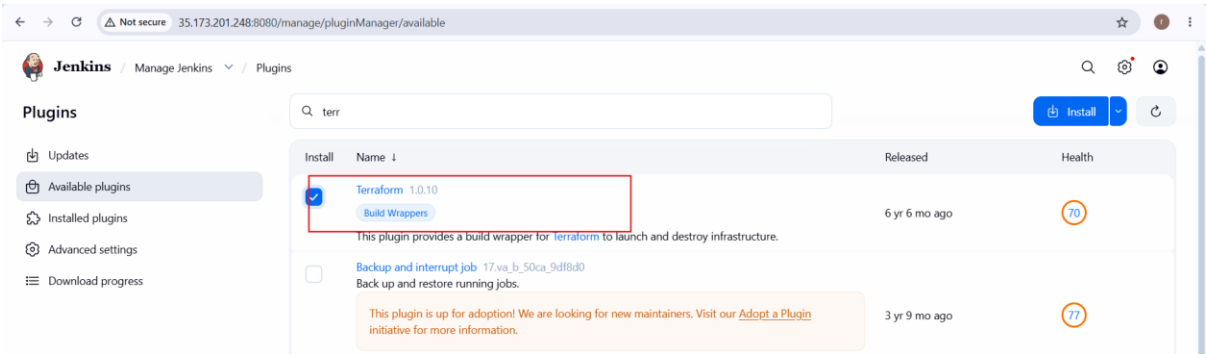
Terraform

The official Jenkins plugin page identifies the plugin ID as:

terraform

and documents installation through the Jenkins plugin manager/CLI.

After installation, restart Jenkins if Jenkins asks you to.



Step 4: Configure Terraform in Jenkins

Go to:

Jenkins Dashboard

↓

Manage Jenkins

↓

Tools

Look for:

Terraform installations

Click:

Add Terraform

Give it a name, for example:

terraform

or:

Terraform-1.13

You can configure the Terraform executable/installation here.

For example:

Name:

terraform

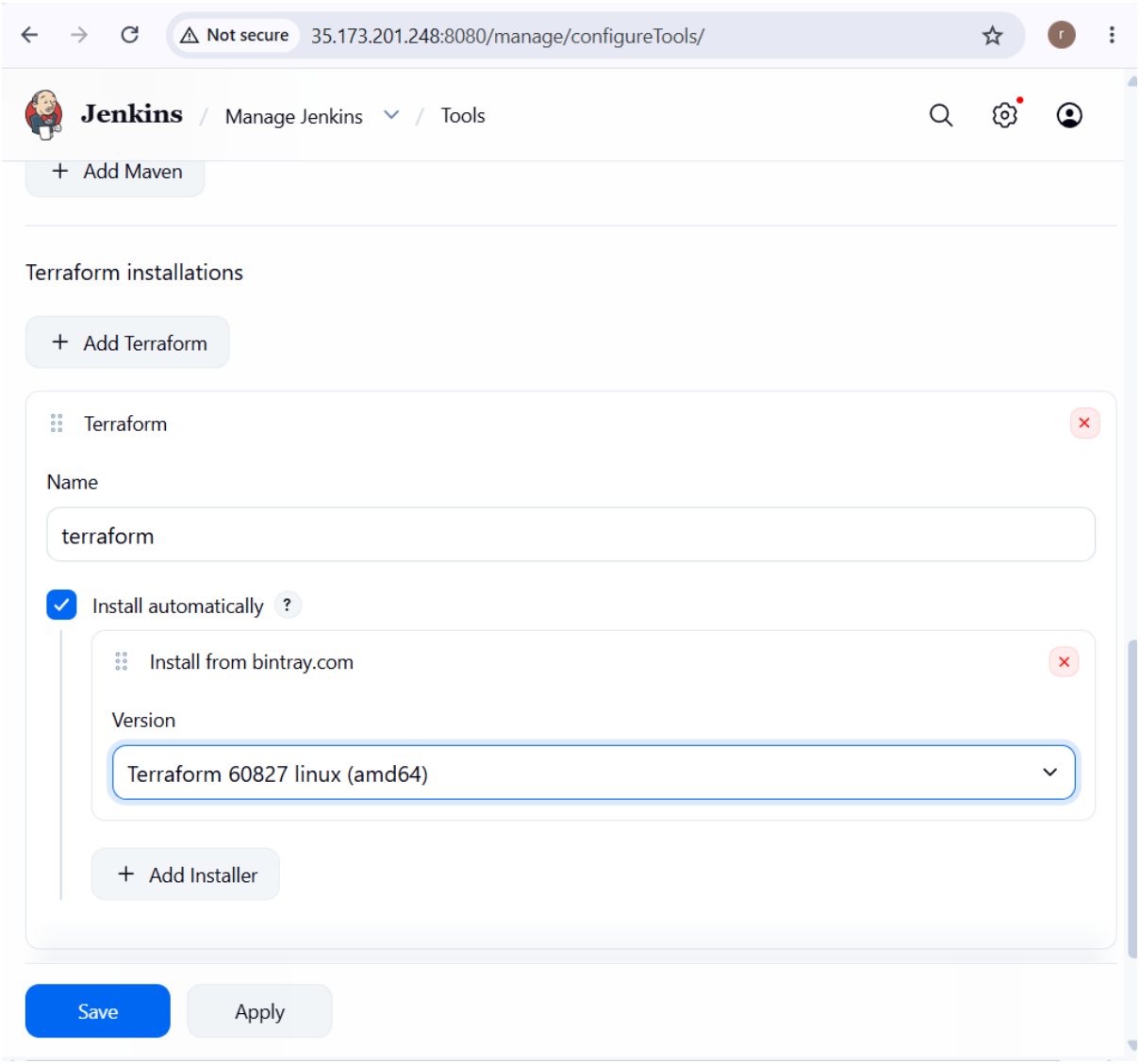
Install automatically:

✓

Or, if Terraform is already installed on your Jenkins server, configure Jenkins to use the existing Terraform installation.

Then click:

Save



3.Create one Jenkins job using Maven Project for the code below with two stages:

- **Stage 1: Git clone**

- **Stage 2: Maven Compilation Code: https://github.com/betawins/java-Working-app.git**

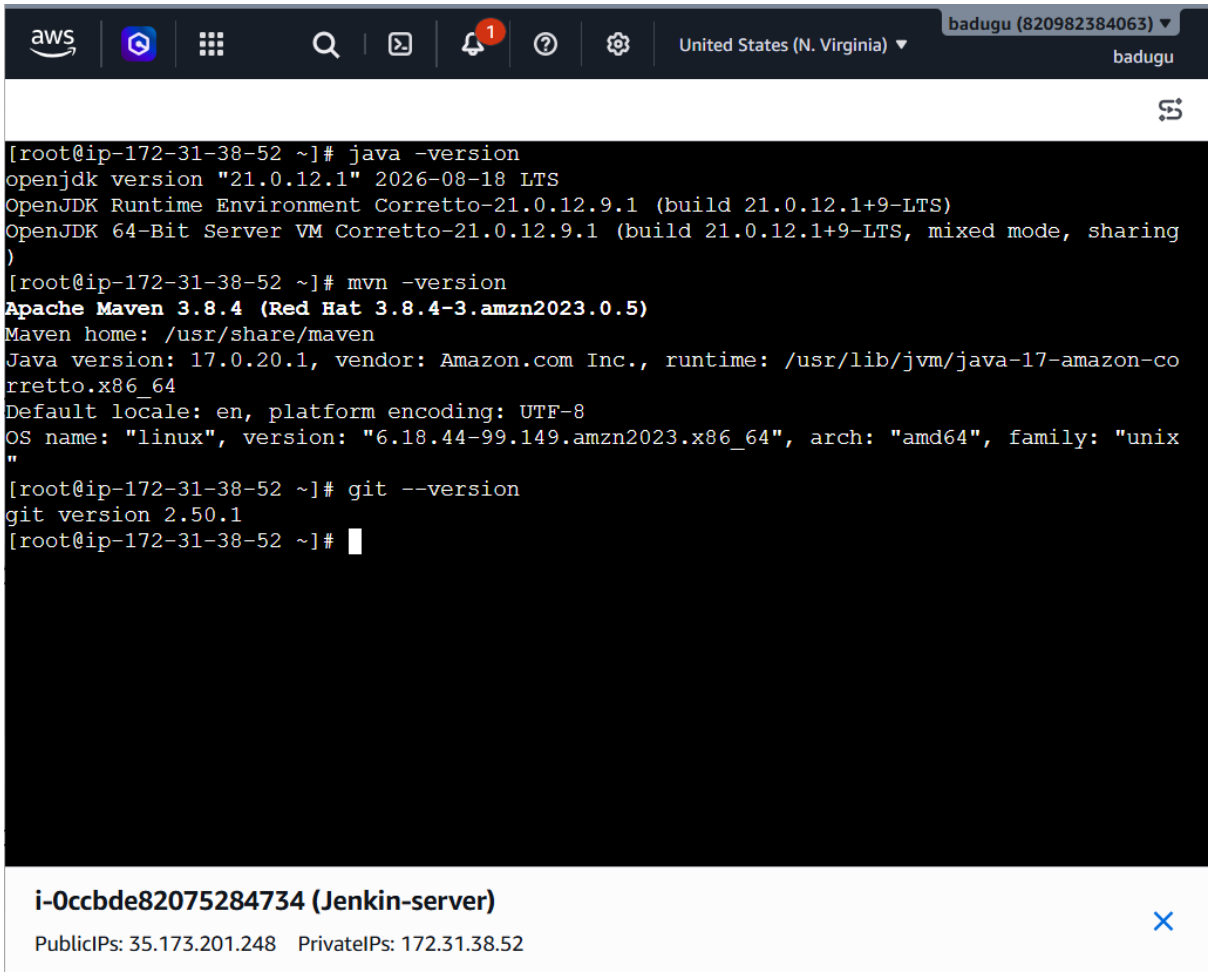
1. First verify Maven on Jenkins

SSH into your Jenkins server and run:

java -version

mvn -version

git --version



2. Configure Maven in Jenkins

Go to:

Jenkins Dashboard

↓

Manage Jenkins

↓

Tools

Find:

Maven installations

Click:

Add Maven

Configure:

Name: Maven-3.8.4

You can either allow Jenkins to install Maven automatically or point it to an existing Maven installation.

For example:

Maven installations

Name:

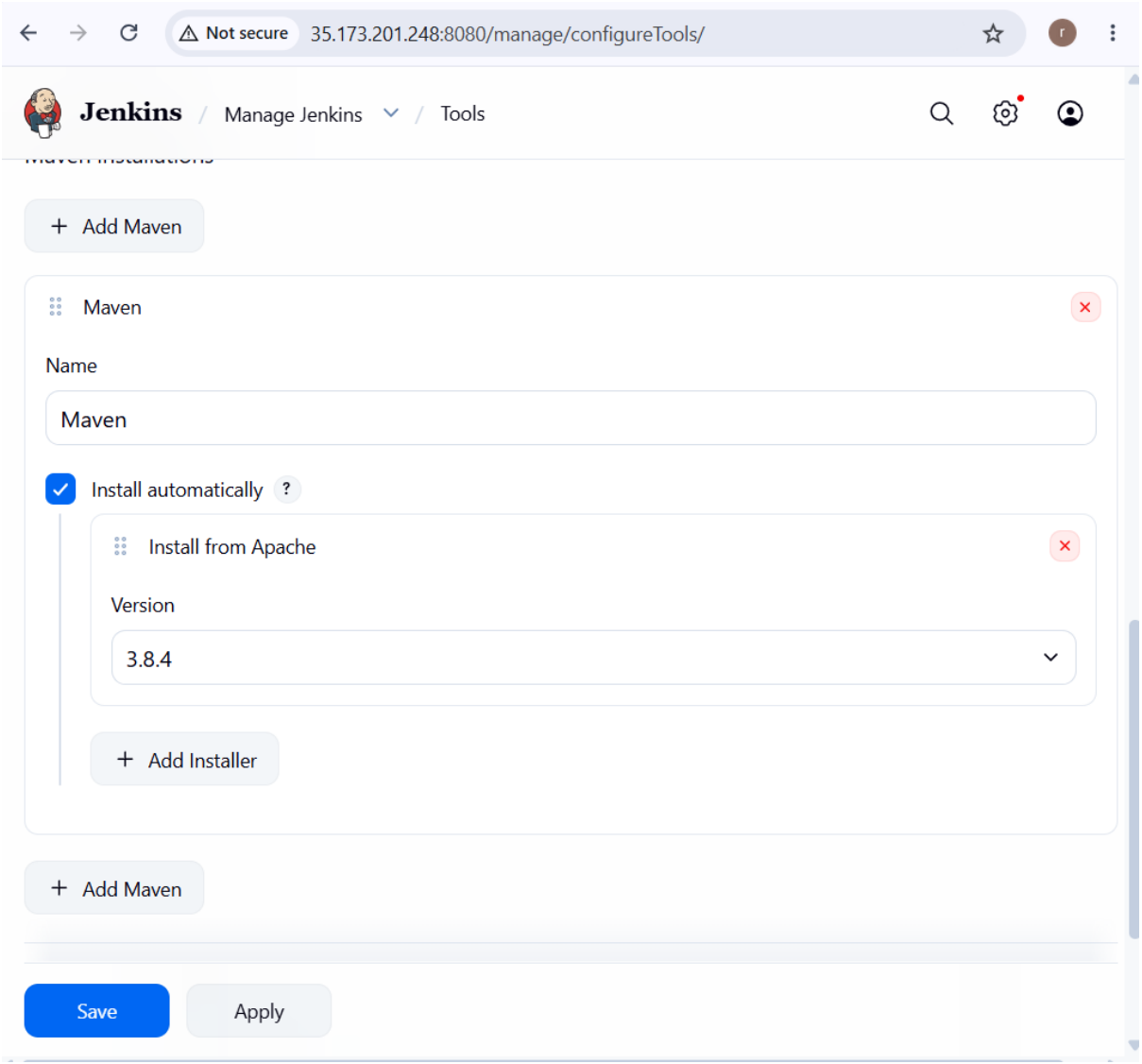
Maven-3.8.4

Version:

3.8.4

Click:

Save



3. Configure Git

Go to:

Manage Jenkins

↓

Tools

↓

Git installations

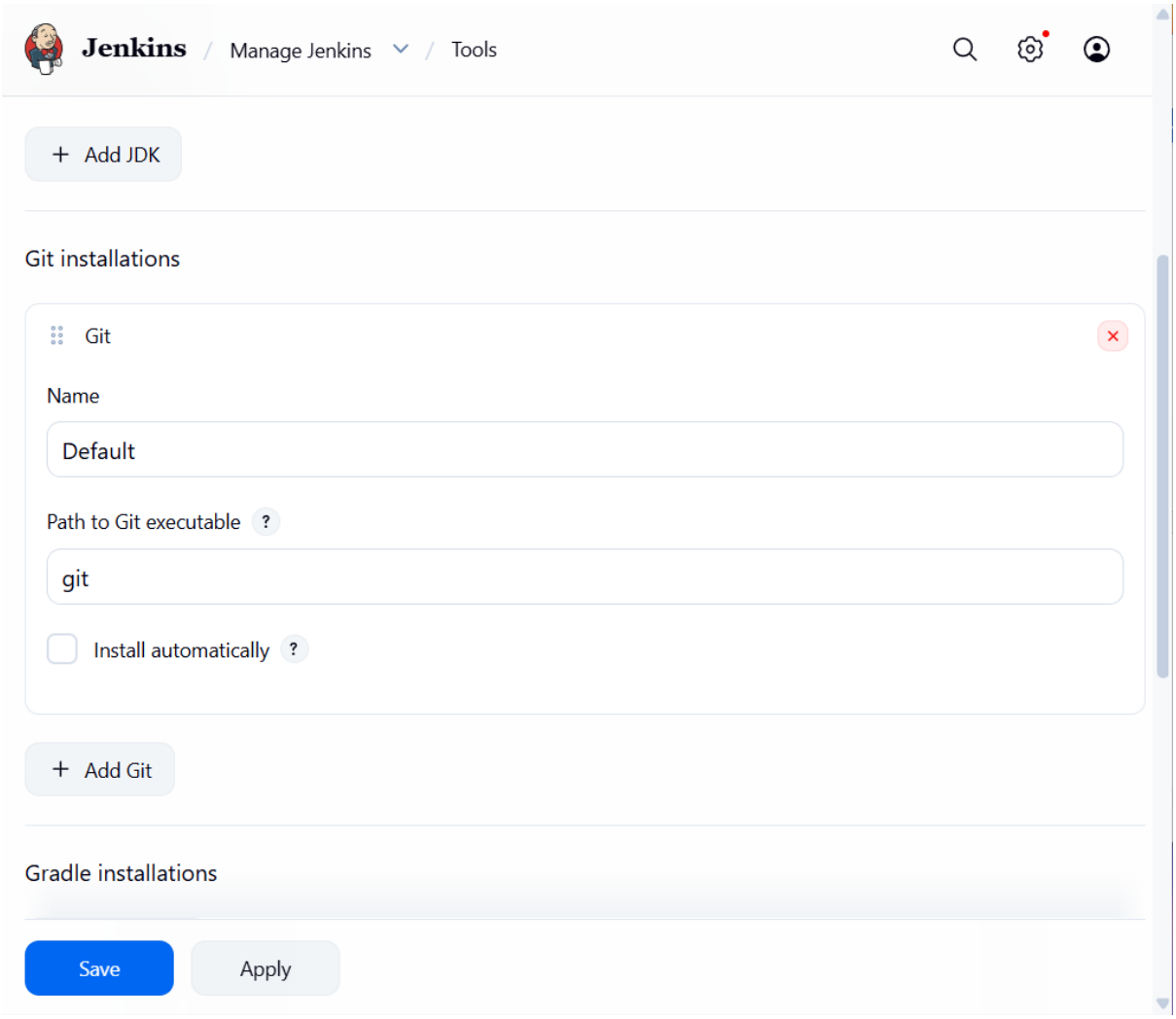
Make sure Git is configured.

For example:

Name: Default

Path to Git executable: git

If Jenkins can find Git automatically, you can leave the default configuration.



4. Create the Maven Project

Go to:

Jenkins Dashboard

↓

New Item

Enter:

Name:

java-working-app

Select:

Maven project

Then click:

OK

You now have a **Freestyle Maven Project**.

Go to:

Manage Jenkins → Plugins → Available plugins

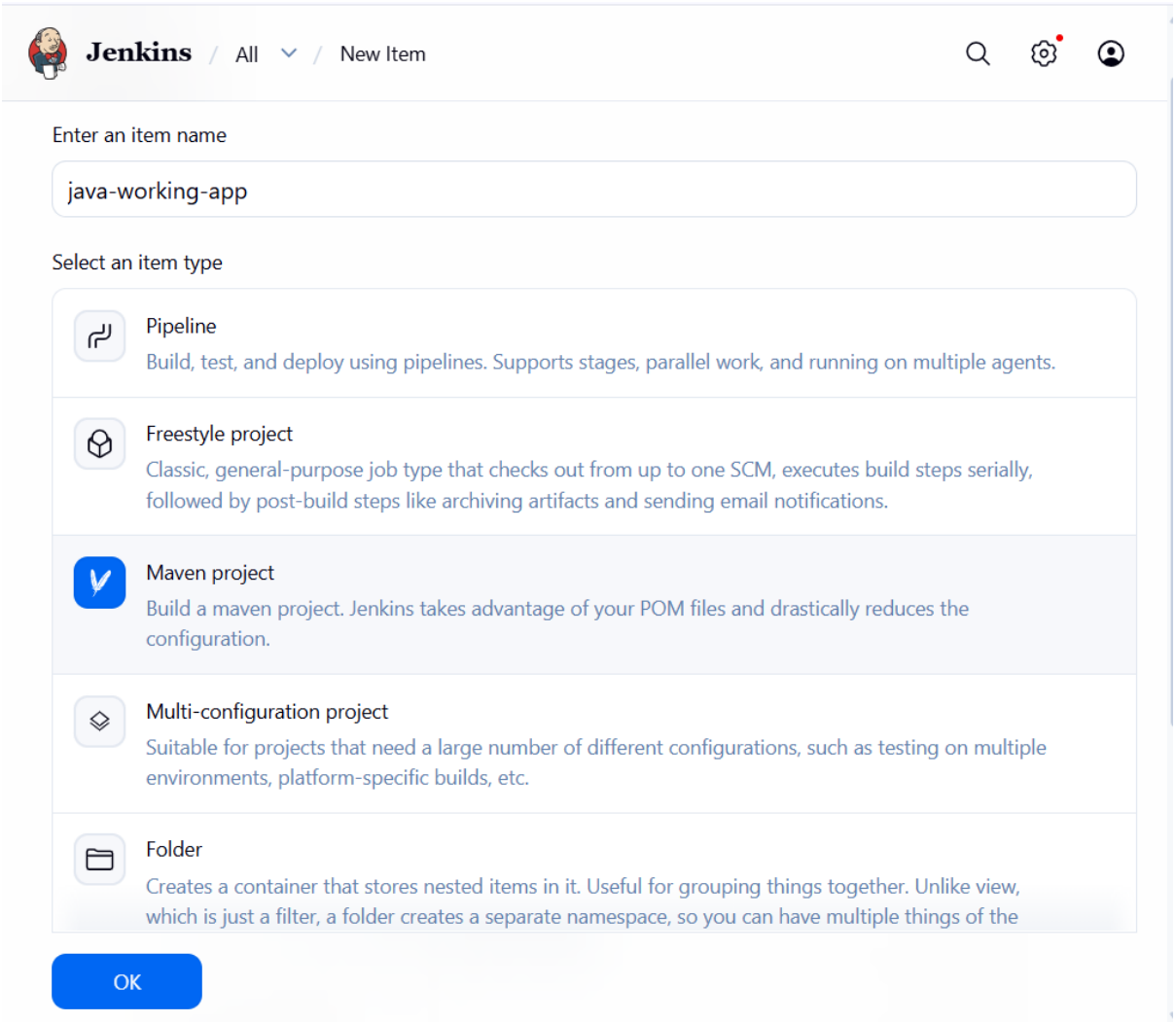
Search for:

Maven Integration

Install:

Maven Integration plugin

Then **restart Jenkins**.



5. Stage 1 — Git Clone

In the job configuration, find:

Source Code Management

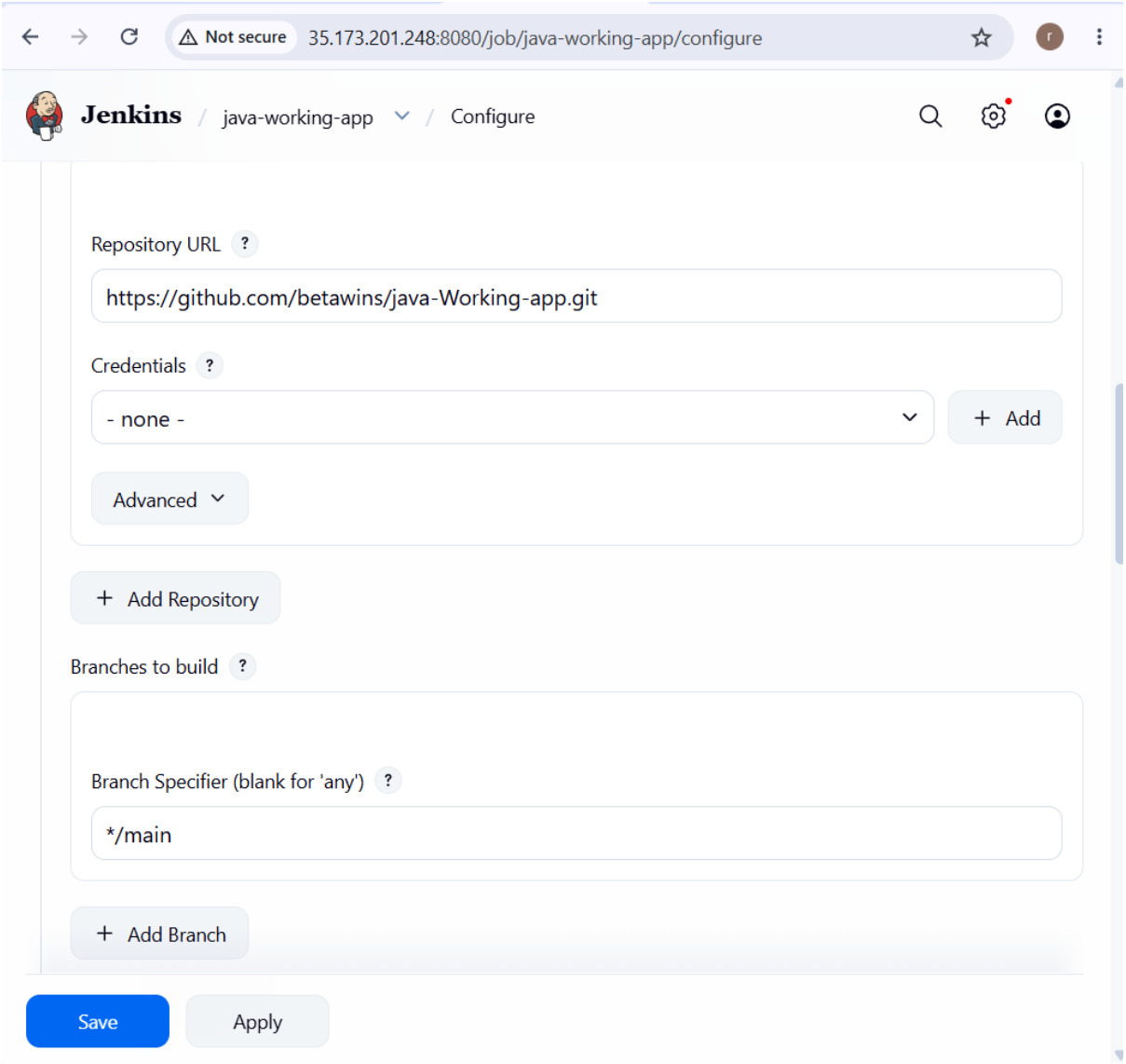
Select:

Git

Enter:

Repository URL:

https://github.com/betawins/java-Working-app.git



6. Stage 2 — Maven Compilation

Now scroll down to:

Build

Because you selected **Maven project**, Jenkins provides Maven build configuration.

Configure:

Root POM:

pom.xml

Then:

Goals and options:

clean compile

So your configuration should look like:

Root POM:

pom.xml

Goals and options:

clean compile

What does this do?

mvn clean compile

clean:

Deletes the previous target directory

compile:

Compiles Java source code

So:

pom.xml

↓

Maven

↓

Download dependencies

↓

Compile Java source code

↓

target/classes

7. Save the Job

Click:

Save

Then click:

Build Now

8. Expected Jenkins execution

You should see something similar to:

Started by user Rajkumari

Building in workspace:

/var/lib/jenkins/workspace/java-working-app

[Git] Cloning repository

https://github.com/betawins/java-Working-app.git

Checking out Revision xxxxx

[INFO] Scanning for projects...

[INFO] --- maven-clean-plugin ---

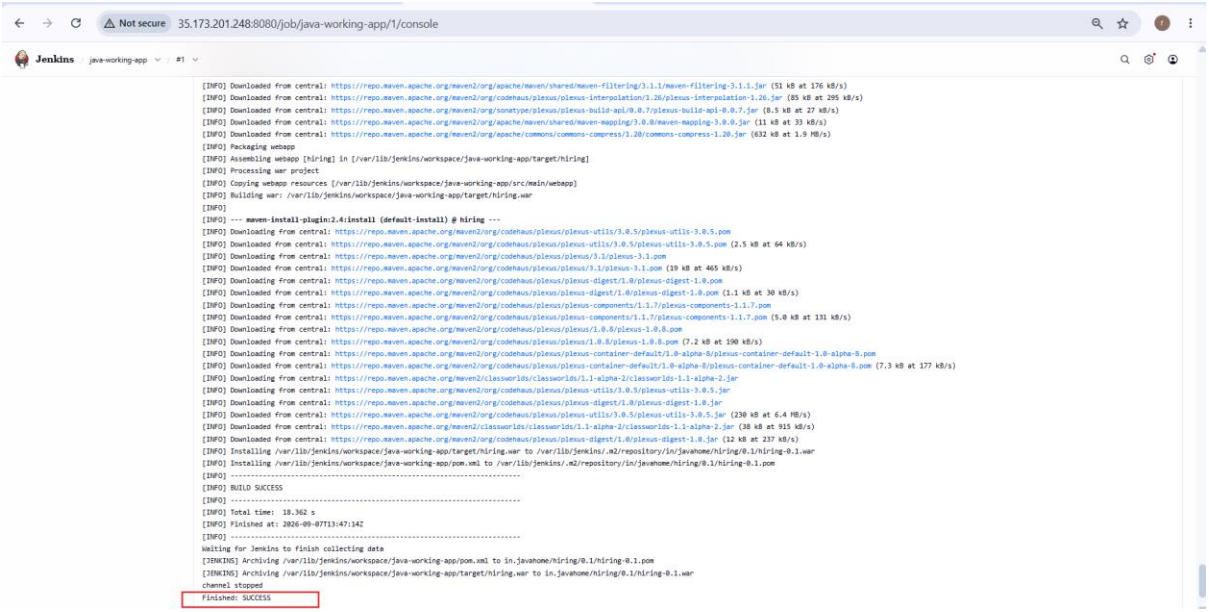
[INFO] Deleting target

[INFO] --- maven-compiler-plugin ---

[INFO] Compiling Java sources

[INFO] BUILD SUCCESS

Finished: SUCCESS



4.Use the same code and create a parameterized job in Jenkins with:

- **Stage 1: Git clone**
- **Stage 2: Maven CompilationCode: https://github.com/betawins/java-Working-app.git**

1. Create the job

Go to:

Jenkins Dashboard

↓

New Item

Enter:

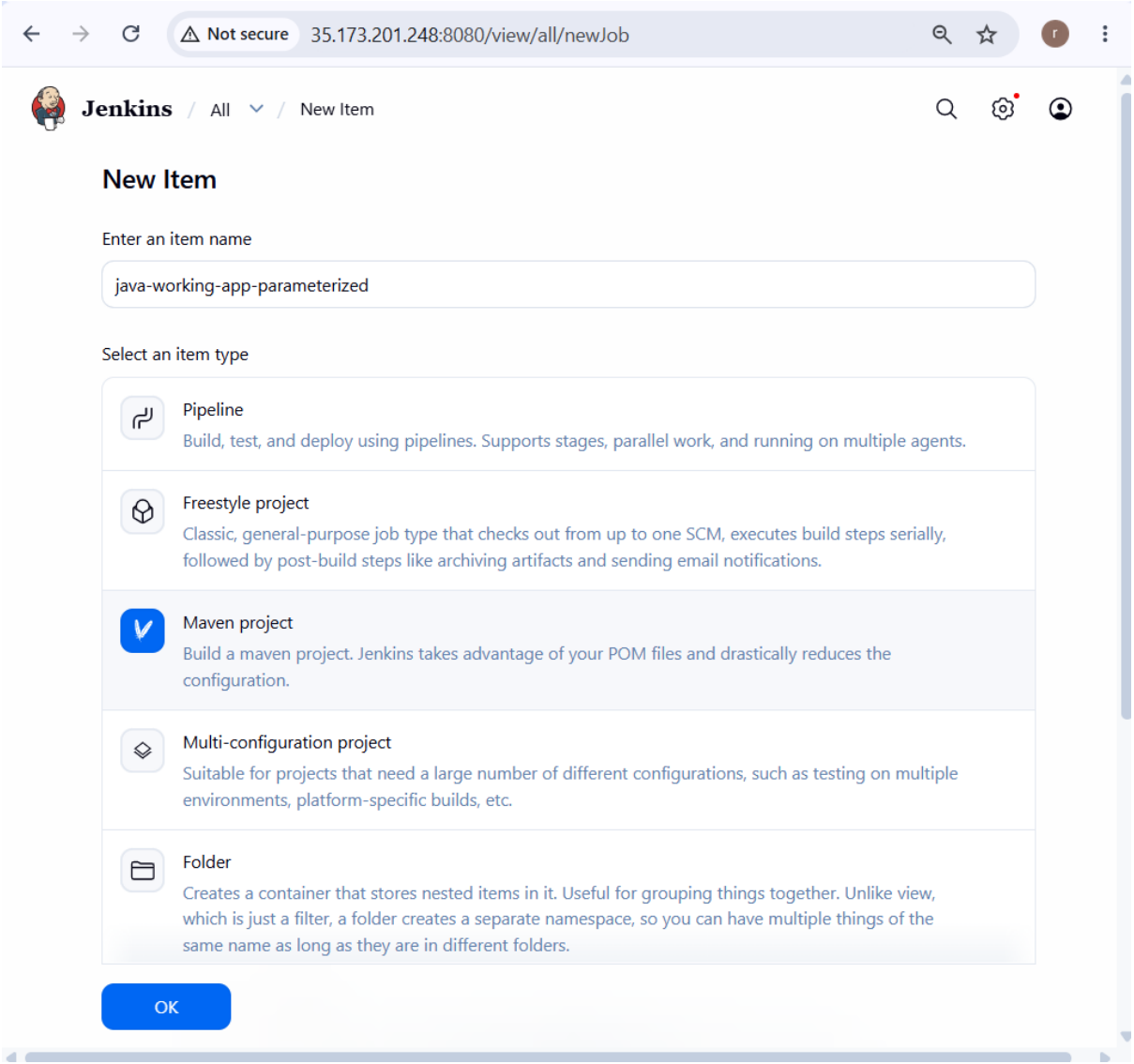
Name:

java-working-app-parameterized

Select:

Maven project

Click **OK**.



2. Enable parameterization

In the job configuration, find:

General

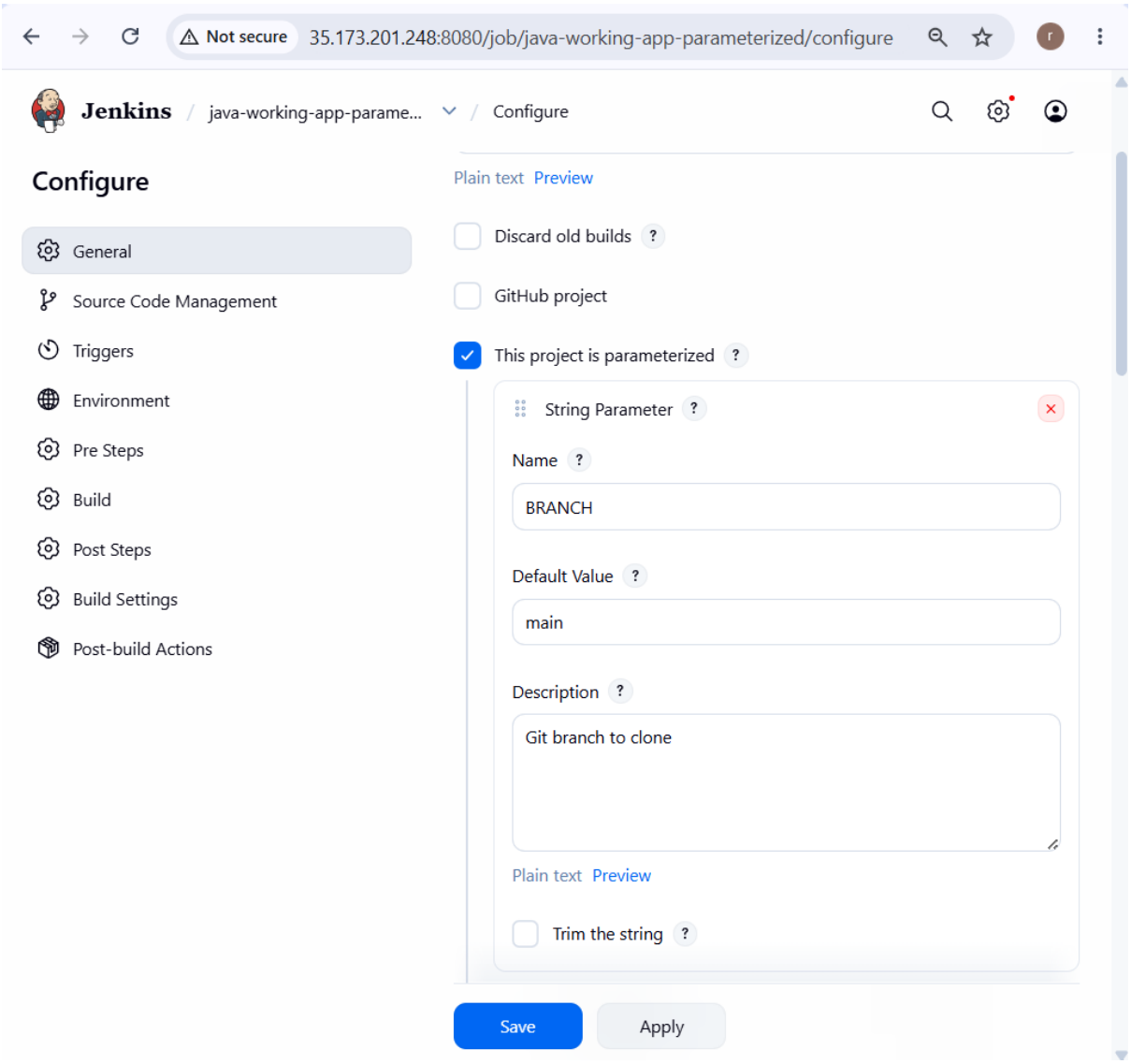
Select:

☑ This project is parameterized

Now click:

Add Parameter

We will create a **String Parameter** for the Git branch.



4. Create MAVEN_GOAL parameter

Again:

Add Parameter

↓

String Parameter

Configure:

Name:

MAVEN_GOAL

Default Value:

clean compile

Description:

Maven goals to execute

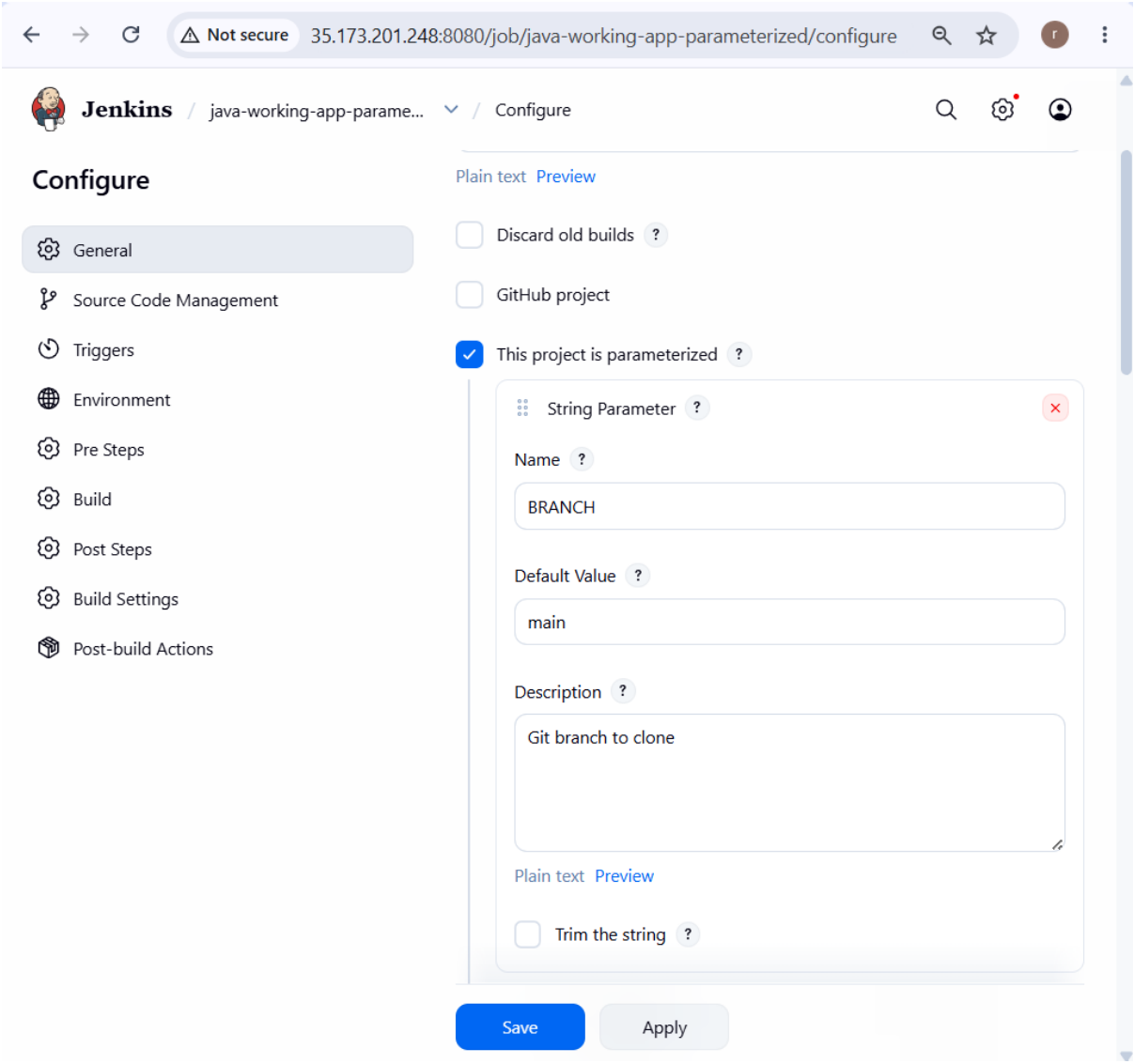
Now you have:

BRANCH

main

MAVEN_GOAL

clean compile



5. Configure Stage 1 — Git Clone

Go to:

Source Code Management

Select:

Git

Repository URL:

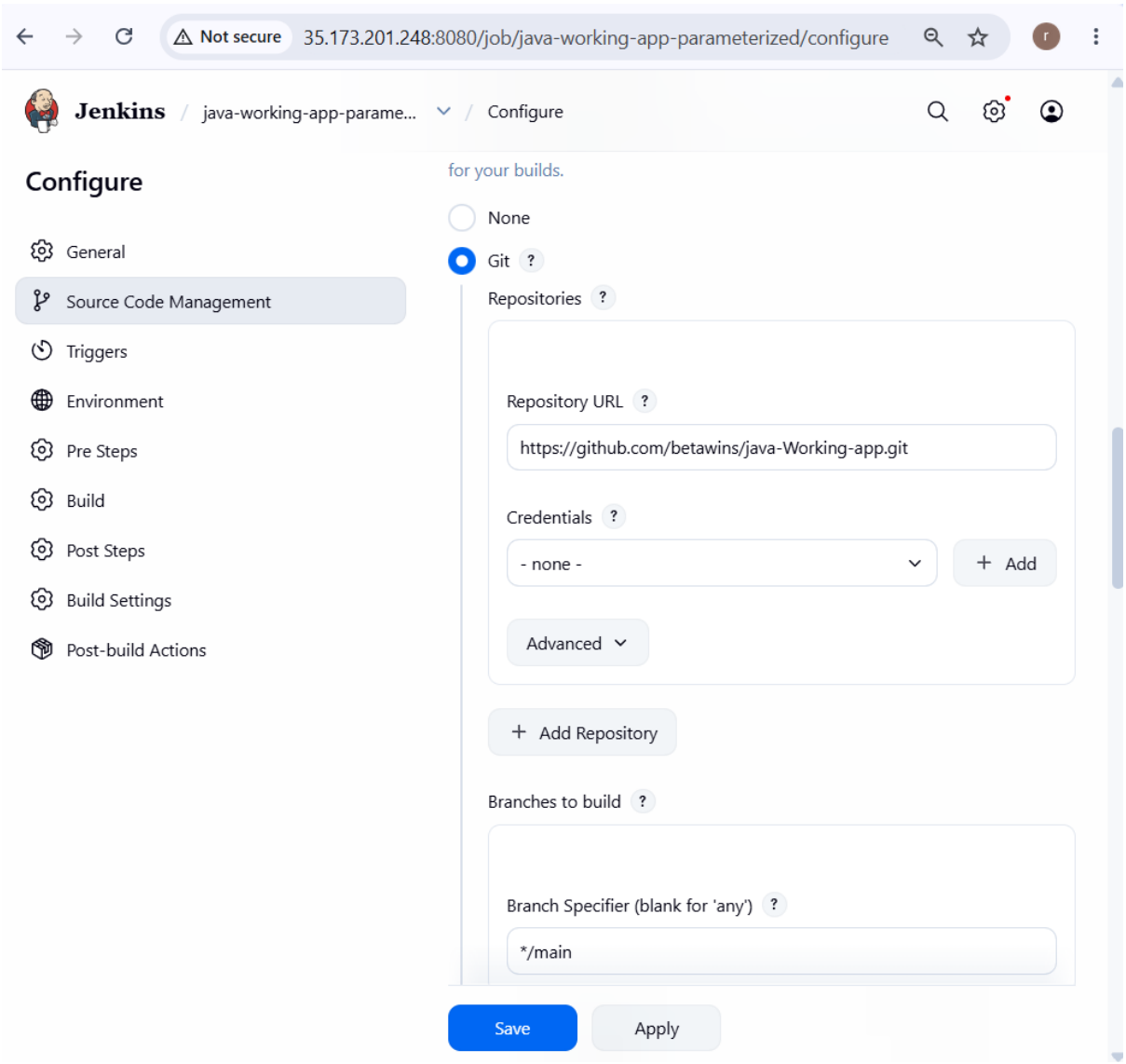
https://github.com/betawins/java-Working-app.git

For **Branch Specifier**, instead of hardcoding:

*/main

use:

*/${BRANCH}



6. Configure Stage 2 — Maven Compilation

Go to:

Build

Because this is a **Maven project**, configure:

Root POM

pom.xml

Goals and options

Use:

${MAVEN_GOAL}

So:

Root POM:

pom.xml

Goals and options:

${MAVEN_GOAL}

When you select the default:

MAVEN_GOAL = clean compile

Jenkins executes:

mvn clean compile

6. Configure Stage 2 — Maven Compilation

Go to:

Build

Because this is a **Maven project**, configure:

Root POM

pom.xml

Goals and options

Use:

${MAVEN_GOAL}

So:

Root POM:

pom.xml

Goals and options:

${MAVEN_GOAL}

When you select the default:

MAVEN_GOAL = clean compile

Jenkins executes:

mvn clean compile

7. Configure Maven installation

Make sure Jenkins has Maven configured.

Go to:

Manage Jenkins

↓

Tools

↓

Maven installations

For example:

Name:

Maven-3.8.4

Then in your Maven project configuration select:

Maven Version:

Maven-3.8.4

8. Save

Click:

Save

Now instead of **Build Now**, you should see:

Build with Parameters

Click it.

9. Run with parameters

You should see:

Build with Parameters

BRANCH

```
┌──────────────────────────────────────────────────────┐
│ main                                                 │
└──────────────────────────────────────────────────────┘
```

MAVEN_GOAL

```
┌──────────────────────────────────────────────────────┐
│ clean compile                                        │
└──────────────────────────────────────────────────────┘
```

[Build]

Leave the defaults:

BRANCH = main

MAVEN_GOAL = clean compile

Click:

Build

10. Jenkins execution

Stage 1 — Git Clone

Jenkins effectively performs:

Repository:

https://github.com/betawins/java-Working-app.git

Branch:

main

and checks the code out into:

/var/lib/jenkins/workspace/java-working-app-parameterized

Stage 2 — Maven Compilation

Jenkins reads:

pom.xml

and executes:

mvn clean compile

The flow is:

pom.xml

↓

Maven

↓

clean

↓

compile

↓

Java source files

↓

target/classes

If everything is successful:

BUILD SUCCESS

11. Try changing the parameter

Now click:

Build with Parameters

You can change:

BRANCH = development

and:

MAVEN_GOAL = clean package

Then Jenkins will execute approximately:

Git Clone

↓

development branch

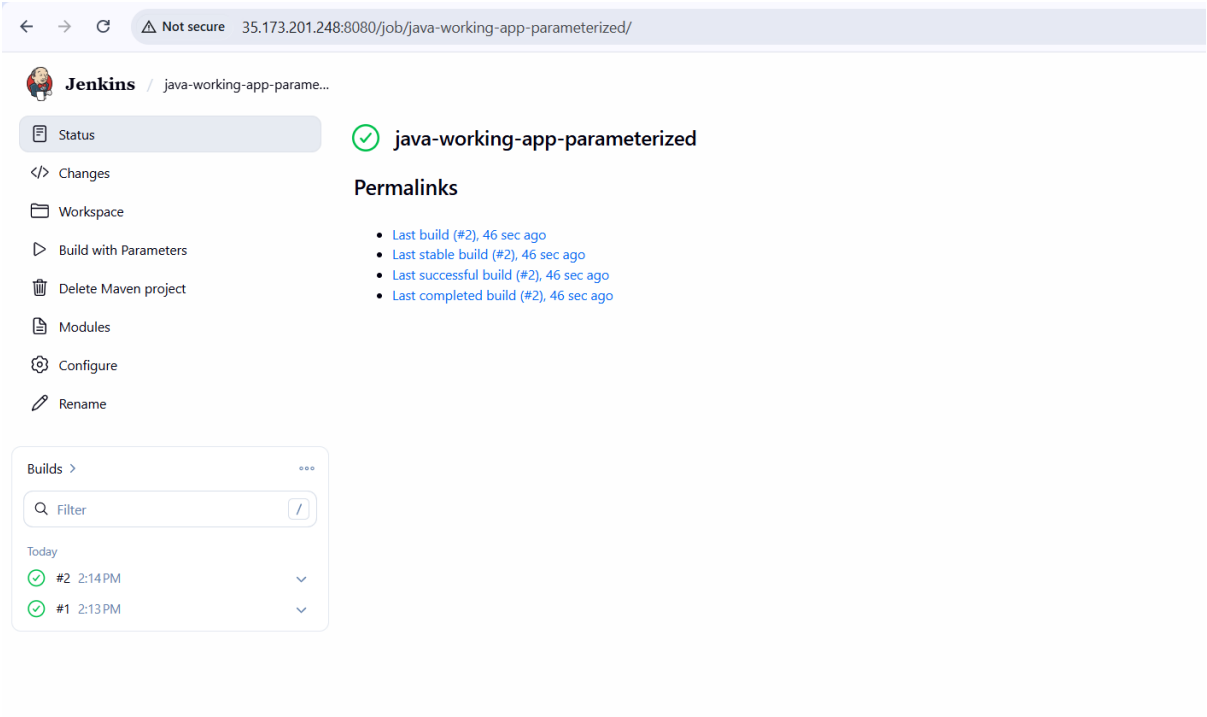
↓

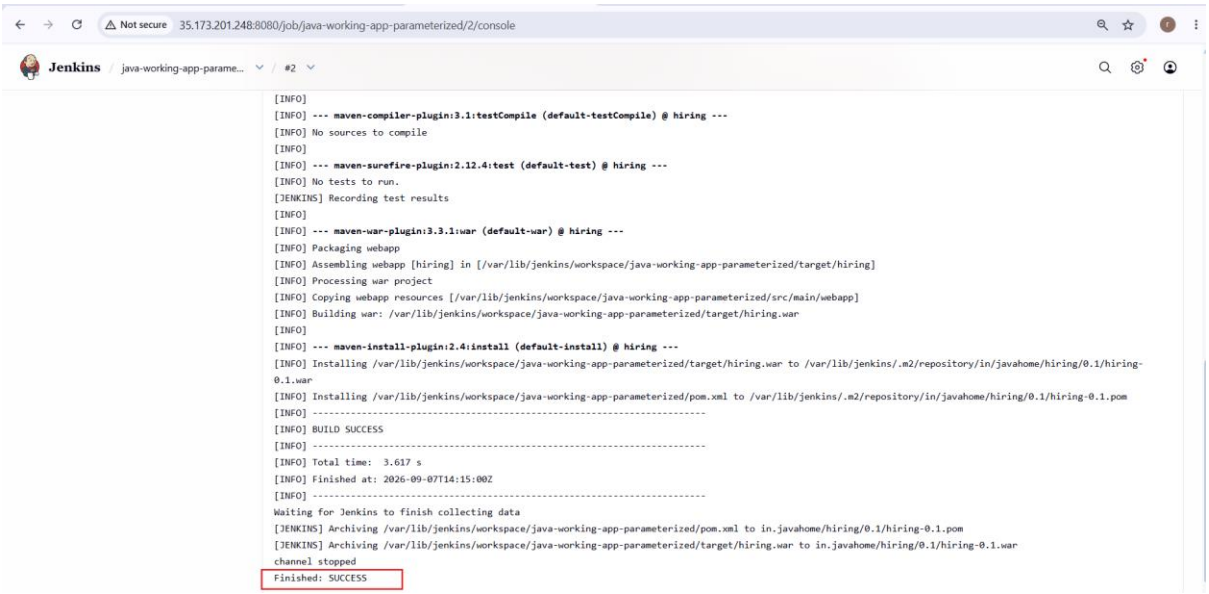
Maven

↓

mvn clean package

So the same Jenkins job can be reused for different branches and Maven operations.





5.What are the global variables in Jenkins?

In Jenkins, **global variables** are variables that are available across Jenkins jobs or throughout a Pipeline, depending on how they are defined.

There are two common meanings of "global variables" in Jenkins:

1. Built-in Jenkins environment variables

Jenkins automatically provides many environment variables.

Examples:

Variable	Meaning	Example
BUILD_NUMBER	Current build number	25
BUILD_ID	ID of the current build	25
BUILD_URL	URL of the build	http://jenkins:8080/job/demo/25/

Variable	Meaning	Example
JOB_NAME	Name of the Jenkins job	java-working-app
WORKSPACE	Workspace directory	/var/lib/jenkins/workspace/java-working-app
NODE_NAME	Jenkins node/agent name	built-in
JENKINS_HOME	Jenkins home directory	/var/lib/jenkins
GIT_COMMIT	Git commit checked out	a82f91...
GIT_BRANCH	Git branch	origin/main

You can see them in a Pipeline using:

```
pipeline {
   agent any

   stages {
      stage('Environment') {
         steps {
            sh 'echo "Job: $JOB_NAME"'
            sh 'echo "Build: $BUILD_NUMBER"'
            sh 'echo "Workspace: $WORKSPACE"'
         }
      }
```

```
    }
}
```

---

2. User-defined global variables

You can also define variables that you want Jenkins to make available broadly.

For example:

JAVA_HOME

MAVEN_HOME

TOMCAT_HOME

These can be configured through Jenkins or the operating-system environment.

For example:

MAVEN_HOME=/opt/maven

JAVA_HOME=/usr/lib/jvm/java-17

Then a Jenkins job can use them.

Environment variables vs global variables

A useful way to remember this:

```
Jenkins
  |
  +-- Built-in environment variables
  |      |
  |      +-- BUILD_NUMBER
  |      +-- JOB_NAME
  |      +-- WORKSPACE
  |      +-- BUILD_URL
```

|

+-- User-defined variables

|

+-- JAVA_HOME

+-- MAVEN_HOME

+-- TOMCAT_HOME

In your Maven job

For your java-Working-app project, you could use:

JOB_NAME

BUILD_NUMBER

WORKSPACE

GIT_BRANCH

GIT_COMMIT

For example:

echo "Job Name: $JOB_NAME"

echo "Build Number: $BUILD_NUMBER"

echo "Workspace: $WORKSPACE"

echo "Git Branch: $GIT_BRANCH"

Global variables in Jenkins are variables that provide information or configuration values that can be accessed by Jenkins jobs or Pipeline executions. Jenkins provides built-in environment variables such as BUILD_NUMBER, JOB_NAME, and WORKSPACE, and administrators can also define custom environment variables for use across jobs.